

How this analysis works

How the analysis is computed

Each condition ran the same 140-agent population through the same simulation, differing only in what memory the agents carry between conversations. Every exchange between two agents is logged as one row: who talked to whom, what each believed before, and what each believed after.

Exchange cadence in the delivered data

Each agent exchanges once per simulated hour, not once per day: 100,800 rows over 30 days works out to 140 agents x 24 hours x 30 days. The raw 'iteration' column in the delivered file is a running exchange counter, not a day index. The import step (scripts/import_condition_results.py) reindexes it to the day number (iteration = day_id) before this analysis runs, preserving the original counter as 'exchange_id'.

Four things get computed, per condition:

1. Entropy: how spread out opinions are across the population on a given day. High means diverse, low means the population is converging on one view. Computed from a full-day population snapshot: each agent's most recent stance as of the end of that day, not a single hourly exchange.
2. Left-leaning share: percent of the population holding a LEFT or FAR LEFT stance, at the start and end. Tracks directional drift specifically, separate from overall diversity. Same day-level snapshot as entropy.
3. Effective clusters: an estimate of how many distinct opinion groups remain at the end. Near 1 means everyone converged to the same view. Near 5 means the population stayed spread

How this analysis works (continued)

across the full range. Computed as $1 / \sum(p_i^2)$, where p_i is each stance's share of the day-level population snapshot.

This was previously computed by counting every hourly exchange separately rather than resolving each agent to a single end-of-day stance first; that undercounted agreement and inflated the apparent number of clusters. Fixed before any of the numbers in the results document were produced.

4. Acceptance rate by stance distance: the probability an agent shifts toward its partner's stance, split out by how far apart the two agents started. Computed on every individual hourly exchange, not the day-level snapshot, since this is about single conversations, not population state. Sample sizes range from about 1,600 to 37,500 exchanges per cell for stance distances of 1 to 3 steps; the ± 4 step cells have only 17 to 45 exchanges each and should not be treated as reliable.

Data verification performed before producing results

Day 1 coverage: confirmed all 140 agents appear at least once on day 1 in every condition, so the first-day entropy and left-share snapshot is a complete population read, not a partial one.

Row-order assumption: the day-level 'most recent stance' resolution assumes rows are already in chronological order within a day. This holds because the import step never reorders rows, but it is not independently re-verified from the `sim_clock` column; noted here as a documented precondition, not a proven invariant.

Open item: exchange cadence

Open item: exchange cadence needs source confirmation

The delivered data shows an hourly exchange cadence (see previous page). This differs from the day-based, once-per-agent-per-day loop this repository's own `src/simulation.py` implements. Rossetti mentioned fixing 'a few minor issues' before the DGX run; the actual code diff has not been shared or pulled into this repo yet.

What the data alone can confirm: within a day, `hour_id` runs 0 through 23, and all 140 agents appear once per `hour_id`, in order, before the hour advances. This is consistent with a nested day/hour loop where every agent exchanges once per simulated hour, 24 times a day. This is an inference from the row shape and ordering, not a confirmed read of the actual loop structure.

What is not yet known: whether `CLOCK_ADVANCE_HOURS` changed, whether checkpointing cadence changed to match, and whether partner selection still draws uniformly at random the way the day-based loop did. None of this affects whether the numbers in the results document are computed correctly from the data as delivered, but it does affect how the methodology section of any eventual paper should describe the loop. Ask for the diff before writing that section.

Conditions and implementation

This document explains the four memory conditions used in the ablation, what distinguishes them mechanically, and how they connect to the research question in `hypothesis_and_scope.md`. It is reference material, not a paper section.

WHY FOUR CONDITIONS

The research question is whether persistent, structured, decaying memory changes the selective-agreement pattern LLM agents show in pairwise debate, relative to a memoryless baseline (Cau et al. 2025, LODAS framework). The hypothesis is more specific than "memory helps": it claims that memory of a partner's stated beliefs (theory-of-mind memory) reduces unconditional agreement more than memory of facts alone, and that combining facts, theory-of-mind, and self-belief tracking produces the most stable opinions over time.

Testing that claim requires isolating what kind of memory is doing the work, not just whether memory is present. Four conditions do this: one baseline with no memory at all, and three memory conditions that each turn on a different combination of content types. All four conditions share the same population, topic, seed, and simulation length; memory content is the only variable.

NO_KG: NO MEMORY (BASELINE)

Conceptually, this is the memoryless condition. An agent enters every exchange with no record of anything said in a prior exchange, by itself or by any partner.

In code, `no_kg` uses `NullMemory`, one of the two concrete implementations of the abstract `AgentMemory` interface in `src/memory.py`. Both of `NullMemory`'s methods, `absorb()` and `get_context()`, are no-ops. Nothing is extracted after an exchange, and nothing is injected into the next prompt. Which memory class gets instantiated is decided once, at startup, in `simulation.py::_build_memory()`; no other file branches on memory condition, so this is the only place the difference between conditions originates.

Conditions and implementation (continued)

GENERAL_ONLY: FACTS

Conceptually, this condition gives an agent memory of factual content raised in conversation, general claims about the topic, without any record of what a specific partner believes.

In code, `general_only` uses `GhostMemory`, the second concrete implementation of `AgentMemory`. `GhostMemory` takes a dimensions list that determines which KG dimensions are active; for `general_only` this list is `["general"]`. On `absorb()`, `GhostMemory` extracts triplets via an LLM call, one per active dimension, so with only "general" active, only general-dimension triplets are extracted, each written into `GhostKG` through `manager.absorb_content()`. On `get_context()`, `GhostMemory` retrieves relevant prior triplets from `GhostKG` and formats them for injection into the next prompt. Because only the general dimension is active, retrieval can only surface general-dimension triplets.

TOM_ONLY: THEORY OF MIND

Conceptually, this condition gives an agent memory of inferred beliefs about a specific conversation partner, not memory of general facts about the topic.

In code, `tom_only` uses the same `GhostMemory` class as `general_only`, with dimensions set to `["tom"]` instead of `["general"]`. The extraction call, the write path through `manager.absorb_content()`, and the retrieval path through `get_context()` are structurally identical to `general_only`. The only difference is which dimension is active and therefore which triplets get extracted and retrieved.

What "theory of mind" means specifically in this codebase

This is not the general philosophical or cognitive-science sense of theory of mind. In

Conditions and implementation (continued)

this codebase it refers to one specific thing: an inferred belief about what a specific conversation partner believes, extracted as a (subject, predicate, object) triplet and tagged with dimension="tom".

Each triplet produced by extraction is (subject, predicate, object, dimension, round) per `src/triplets.py`. The dimension field is what tags a triplet as "tom" versus "general" versus "beliefs". A "tom" triplet encodes belief-about-partner content (what the partner appears to think); a "general" triplet encodes belief-about-topic content (a fact stated in the exchange, independent of who holds it). The round field records the simulated day the triplet was extracted on; it is retained in the project's own logs for analysis but is stripped before the triplet is handed to GhostKG's `absorb_content()`, which only accepts (subject, predicate, object).

Mechanically, `tom_only` and `general_only` differ in exactly one place: the dimensions list passed to GhostMemory at construction. That list controls which dimension the extraction LLM call is run for, which in turn controls what gets written to GhostKG and what can later be retrieved. Storage and retrieval code paths are identical between the two conditions. The distinction is entirely in what content is tagged and extracted, not in how it is stored, decayed, or retrieved.

FULL_KG: GENERAL + TOM + BELIEFS COMBINED

Conceptually, this condition combines factual memory, theory-of-mind memory about the partner, and the agent's own stated beliefs (self-belief tracking), all at once.

In code, `full_kg` uses GhostMemory with dimensions set to ["general", "tom", "beliefs"]. `absorb()` runs one extraction LLM call per active dimension, so three extraction calls happen per absorbed turn instead of one, each tagged with its own dimension and written to GhostKG separately. `get_context()` retrieves and formats prior triplets across all three dimensions for injection into the next prompt.

Conditions and implementation (continued)

FSRS DECAY APPLIES UNIFORMLY ACROSS ALL THREE KG CONDITIONS

Decay is not a variable across `general_only`, `tom_only`, and `full_kg`. GhostKG applies FSRS (a spaced-repetition scheduling algorithm) to triplet strength, keyed on the simulated clock, which is set via `manager.set_agent_time(agent_id, clock)`, called once per day with the simulated date. Older, less-reinforced triplets retrieve with lower weight over simulated time. This mechanism is identical regardless of which dimensions are active; it operates on whatever triplets exist in GhostKG for a given agent, without regard to whether those triplets came from the general, tom, or beliefs dimension. The `round` field on the project's own Triplet dataclass is not an input to this decay mechanism; decay is driven entirely by `set_agent_time()`, separately from the extraction dimension list. The specific FSRS parameters in use are GhostKG's defaults; this project has not tuned them.

The practical implication: what differs between `general_only`, `tom_only`, and `full_kg` is exclusively which dimensions are extracted and retrieved. The decay curve applied to whatever triplets exist is the same curve in all three conditions.